DMS – Depot Management System: POC Concept Document

Oct 27, 2026 · @Abhiraj Ghosh

Executive summary

The DMS POC proves one loop end to end: a vehicle arrives, the gate registers its requirement, the right shed is alerted at once, loading is confirmed, stock updates itself and the vehicle is checked out.

- **Gate Tab:** registers arriving vehicles and their requirements; checks vehicles out.
- **Shed Tab:** receives incoming-vehicle alerts instantly; marks loading complete.
- **Admin Tab:** full inventory CRUD, live visibility of every vehicle, and in/out logs.
- **Real-time sync:** every status change reaches all tablets within about a second, with no refresh.

Background and POC objectives

The client wants a Depot Management System (DMS) that replaces manual coordination between the gate, the storage sheds and the depot office when vehicles arrive to collect ammunition. They have asked for an initial proof of concept (POC) before committing to a full build, with an estimate of price and timeline.

The POC has four objectives:

1. Show the full vehicle lifecycle (arrival, requirement entry, shed alert, loading, stock update, checkout) working across three tablets.
2. Show that shed users are alerted the moment a requirement is registered, with no phone calls or paper slips.
3. Show that inventory and the in/out log stay correct automatically after every loading.
4. Give the client a link they can open from any location, to judge the concept on real devices.

Success means the client can run a scripted demo of three to five vehicles through the full cycle without manual intervention or data correction.

Users and devices

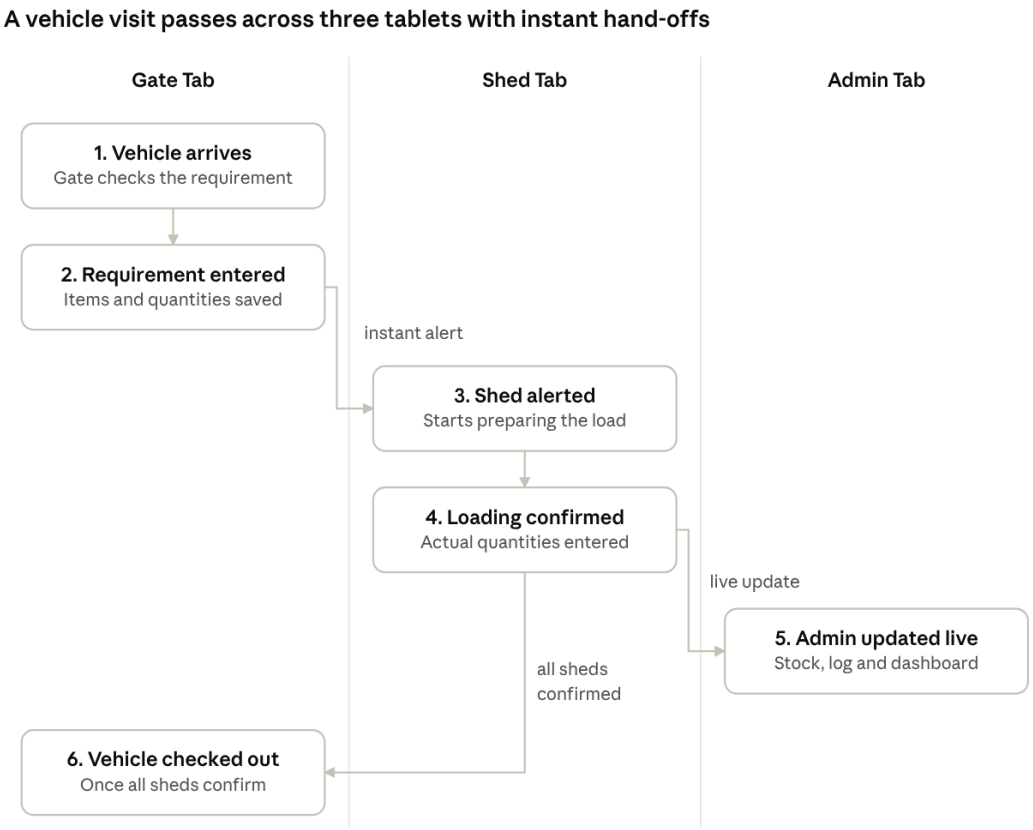
Three tablets, three roles. Each user sees only the screens their role needs.

Tablet	Who uses it	Main actions	Key screens
Admin Tab	Depot administrator / officer in charge	Manage inventory; monitor every vehicle live; review logs	inventory list and editor, in/out log, user management
Gate Tab	Gate security / entry clerk	Register arriving vehicle and its requirement; check vehicle out	New vehicle entry, requirement form, vehicles

Tablet	Who uses it	Main actions	Key screens
			inside, checkout
Shed Tab	Shed storekeeper (one login per shed)	Receive incoming-vehicle alert; prepare load; confirm what was loaded	Incoming queue, load confirmation, shed stock view

The tablets run the system in a browser (installable web app), so no app-store release is needed for the POC.

End-to-end workflow



visit workflow · 6 steps across 3 tablets

The gate registers the requirement, the right shed is alerted at once, and checkout is allowed only after every shed confirms loading. The admin sees each change live.

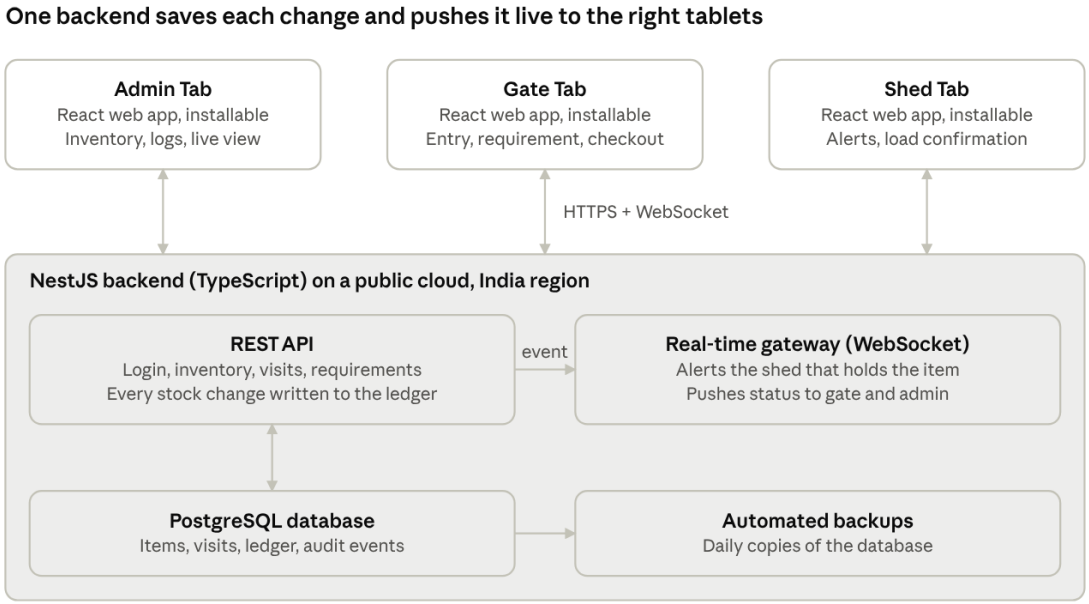
Functional scope of the POC

The POC covers 12 capabilities. Ten are must-haves for the demo; two are small additions that make the demo more convincing.

#	Module	Capability	Tablet	Priority
1	Access	Role-based login (admin, gate, shed); each shed tablet is tied to its shed	All	Must
2	Gate	Register arriving vehicle (vehicle number, driver, unit or party, arrival time)	Gate	Must
3	Gate	Enter requirement as item and quantity lines; warn when stock is short	Gate	Must
4	Routing	Send each requirement line to the shed that holds the item and alert that shed instantly	System / Shed	Must
5	Shed	Incoming-vehicle queue; confirm items and actual quantities loaded	Shed	Must
6	Stock	Deduct stock automatically on loading confirmation and write a ledger entry	System	Must
7	Gate	Check vehicle out once every shed has confirmed loading	Gate	Must
8	Inventory	Create, view, edit and delete items; set category, unit, shed and opening stock	Admin	Must
9	Inventory	Record stock received (Inventory In) and manual adjustments with a reason	Admin	Must
10	Logs	In/out log: which vehicle came, when, what was loaded, who handled it; filter and search	Admin	Must
11	Dashboard	Live view of vehicles inside, status per shed, and current stock levels	Admin	Should
12	Logs	Export the in/out log to CSV or Excel	Admin	Should

A vehicle may need items from more than one shed. The POC handles this by tracking each shed's confirmation separately and allowing checkout only when all are complete.

Architecture and technology



architecture · 3 tablets, 1 backend, 1 database

Each tablet is a browser web app that calls the REST API and keeps a WebSocket open. When a user saves a change, the API writes it to the database in one transaction, then the gateway pushes the event to the tablets that need it: the shed that holds the item, the gate and the admin.

Layer	Proposed choice	Why
Front end	React web app, installable, tablet-first layout	One codebase for all three role screens; no app-store release
Backend	NestJS/Express (Node.js, TypeScript)	Modular structure per domain; built-in WebSocket support
Real-time	WebSockets (Socket.IO)	Targeted pushes: one channel per shed, one for the gate, one for admin
Database	PostgreSQL	Transactions keep stock deductions and ledger entries consistent
Hosting	Public cloud, India region, containerised, HTTPS	Quick to stand up for a POC; easy to move on-premise in Phase 2

Core data model

Seven entities carry the whole workflow. Every stock change is a ledger row, so the inventory number can always be explained and audited.

Entity	Purpose	Key fields
User	Login and role	name, role (admin, gate, shed), shed (for shed users)
Shed	Storage location	name, code
Item	Inventory master	name, category, unit, shed, current stock, reorder level
Visit	One vehicle's trip through the depot	vehicle number, driver, party, status, arrival time, checkout time, gate user
Requirement line	One item and quantity needed on a visit	visit, item, shed, requested qty, loaded qty, status
Stock ledger	Every stock movement (in, out, adjustment)	item, type, quantity, visit (if any), user, timestamp, reason
Audit event	Who did what and when	user, action, entity, timestamp

Visit status moves through: Registered, Awaiting loading, Partly loaded, Loaded, Checked out.